

Phase 6 帳票・PDF 出力 完了報告書

現場 OS Lite / 日報 PDF + 写真台帳 PDF の出力・共有

作成日：2026 年 6 月 24 日 / 目的：現場記録を「提出できる成果物（PDF）」として出力・共有できる

結論：実装・自動検証・CI まで完了 

日報・写真台帳の **2 帳票を PDF 出力** → **OS 共有シートで共有** する機能を、**DB 変更なし・新規画面なし**で追加。flutter analyze=**No issues** / flutter test=**52 件 全合格** / iOS シミュレータビルド成功 / **CI 緑**。実フォントでの日本語 PDF 描画は **ローカルで PNG 目視確認済み**（本書 P.3 に実物プレビュー）。残りは「アプリ内ボタン→共有シート表示」の端末タップ最終確認のみ（アプリはインストール・起動済み）。

1. 実装した機能

機能	内容
日報 PDF 出力	日報詳細の PDF アイコン から、1 件の日報を A4 PDF 化（タイトル「日報」+作業日/天候/作業人数/作成者/更新日時の項目テーブル+作業内容）→ 共有シート。
写真台帳 PDF 出力	写真ギャラリーの PDF アイコン から、現場の写真を 3 列グリッド で台帳 PDF 化（見出し「写真台帳」+現場名、各写真下に撮影日時、自動改ページ）→ 共有シート。
OS 共有・印刷	printing の共有シート（AirDrop/メール/ファイル保存/印刷）に委譲。アプリ内プレビュー画面は作らない（MVP）。
日本語表示	PdfGoogleFonts.notoSansJPRegular を 実行時取得 （アセット同梱なし・取得後キャッシュ）。

2. スcope（実装する / しない）

実装した（MVP）	非対象（後続・対象外）
日報 PDF（1 件→共有）	帳票テンプレートのカスタマイズ/レイアウト編集
写真台帳 PDF（グリッド→共有）	Excel / CSV 出力
OS 標準の共有/保存/印刷	電子署名・押印・ウォーターマーク
現場名・日付・撮影日時などの基本項目	クラウド保存・メール自動送信/月次集計

3. 追加・変更ファイル

区分	ファイル
追加（実装）	lib/features/export/data/pdf_service.dart（ PdfService+PrintingPdfService+pdfServiceProvider） / lib/features/export/application/pdf_export_controller.dart（PdfExportController・autoDispose）
変更（実装）	photos/data/photo_repository.dart（downloadPhoto 追加） / reports/presentation/report_detail_screen.dart（PDF 出力アクション） / photos/presentation/photo_gallery_screen.dart（写真台帳 PDF アクション） / pubspec.yaml（pdf・printing）
追加（テスト）	test/export/fakes.dart（FakePdfService・FakeReportRepository） / test/export/pdf_export_controller_test.dart（5 件） / test/export/pdf_service_smoke_test.dart（1 件）
変更（テスト）	test/photos/fakes.dart（FakePhotoRepository に downloadPhoto）
追加（ドキュメント）	docs/Phase6_帳票 PDF 出力_要件定義.md / docs/Phase6_帳票 PDF 出力_実装仕様書.md / 本完了報告書
変更なし	DB マイグレーション / Supabase 手作業 / 新規画面

※ 自動生成の pubspec.lock と各 OS の generated_plugin_registrant（pdf/printing 登録）も更新。

4. 技術構成

- ・ **PdfService (抽象)** : buildReportPdf / buildPhotoLedgerPdf / sharePdf の 3 メソッド。実装 PrintingPdfService は pdf+printing を使用。フォント取得関数を **注入可能** にし、テストはネット非依存。
- ・ **PdfExportController (AsyncNotifierProvider.autoDispose)** : exportReport(reportId) / exportPhotoLedger(siteId, siteName)。AsyncValue.guard で成否を state に反映し、成功で true。写真 0 枚は専用例外で false。
- ・ **データ取得** : 既存 reportRepositoryProvider / photoRepositoryProvider を流用。PhotoRepository.downloadPhoto(path) のみ追加 (Storage バイト取得・RLS で自社のみ)。**新 Repository なし・DB 変更なし**。
- ・ **UI** : 既存 2 画面 (日報詳細・写真ギャラリー) の AppBar に PDF アクションを追加。生成中はスピナー表示、失敗時 SnackBar。**新規画面なし**。

5. 品質チェック結果

項目	結果	内容
flutter analyze	No issues	警告・エラー 0
flutter test	52 / 52 緑	Phase 6 で新規 6 件 (Controller 5+スモーク 1)。既存 46 件はそのまま緑
iOS ビルド	成功	flutter build ios --simulator --flavor dev。printing は Swift Package Manager で解決
CI (GitHub Actions)	緑	b7603ff : analyze+test を main push で自動実行

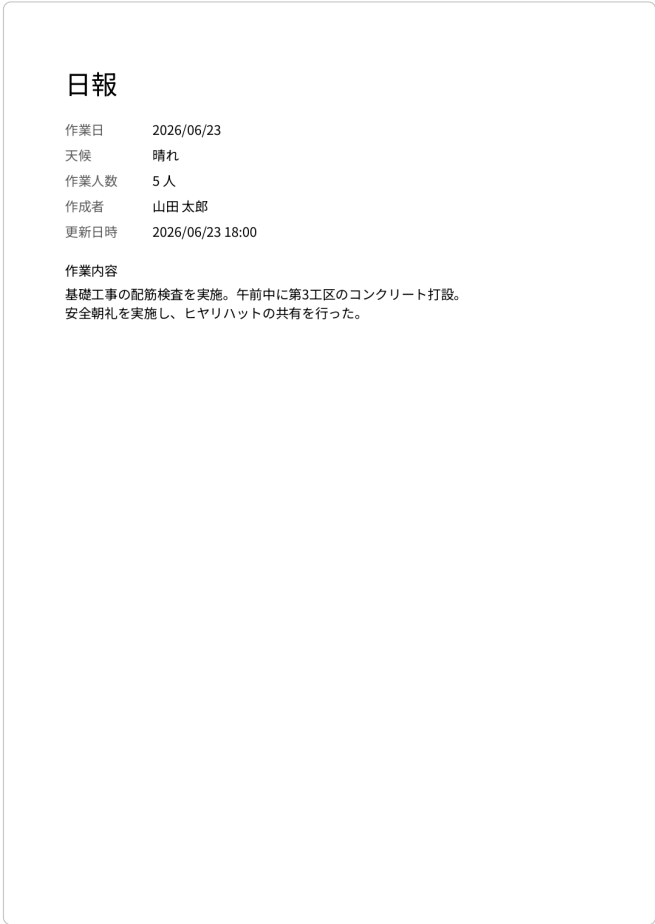
新規テストの内訳

- ・ exportReport : 成功 (true・PDF 生成・共有・ファイル名 日報_2026-06-23.pdf) / 取得失敗 (false・state エラー・共有されない)
- ・ exportPhotoLedger : 写真 2 枚 (true・downloadPhotox2・台帳生成・共有) / 0 枚 (false・生成も共有もしない) / 生成失敗 (false・state エラー)
- ・ スモーク : buildReportPdf が **空でない PDF バイト列** を返す (フォント注入でネット非依存)

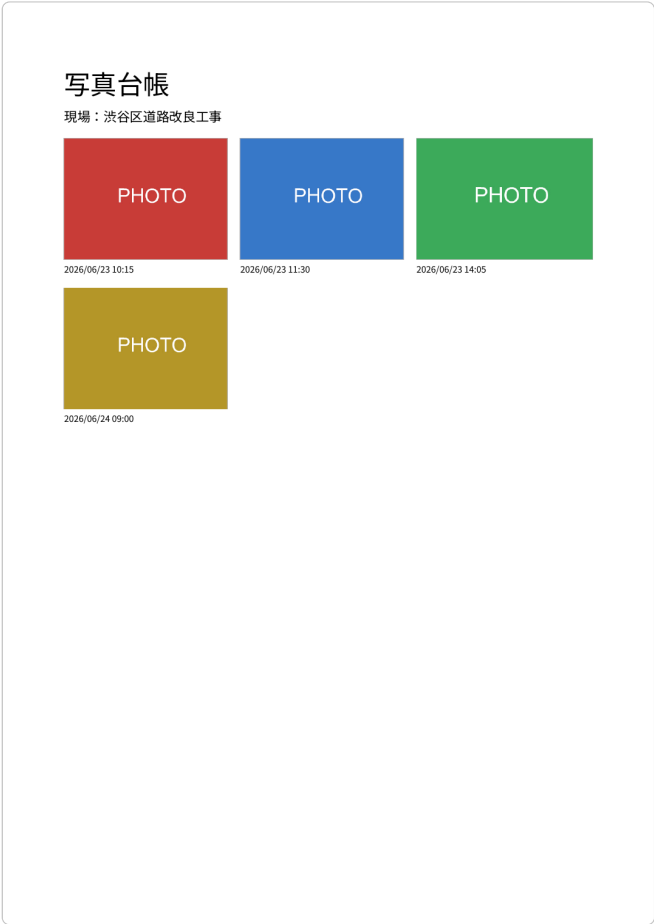
6. 生成 PDF の実物プレビュー（ローカル検証）

実フォント（Noto Sans JP）を埋め込んで実際に生成し、PDF を PNG に変換して目視確認した結果です。日本語が正しく描画され、写真台帳は 3 列グリッド+撮影日時+自動改行 が機能しています。

① 日報 PDF



② 写真台帳 PDF



※ サンプルデータで生成（写真はテスト用の単色画像）。実機では実際の現場写真・日報内容が入ります。

7. 完了条件チェック

#	条件	判定
1	日報 PDF を生成・共有できる	✓
2	写真台帳 PDF を生成・共有できる（現場名・日時付き）	✓
3	既存 RLS 維持（自社データのみ）	✓
4	flutter analyze 成功	✓
5	flutter test 成功	✓
6	dev 実機確認（共有シート表示）	△ 端末タップ最終確認のみ

※ 6 は、PDF 生成・描画・iOS ビルド・CI まで検証済み。残るのは「アプリ内ボタン→OS 共有シートが開く」最終目視のみ（アプリは dev シミュレータにインストール・起動済み）。

8. ROADMAP 更新内容

- ・ Phase 6（帳票・出力）を → 完了（2026-06-24）に更新。
- ・ ★現在地を「Phase 1～4 + 6 完了」に更新。次候補は Phase 5（通知・連携）／Phase 7（組織・権限）。
- ・ 実装メモ追記：PDF は lib/features/export/、共有は Printing.sharePdf、日本語フォントは注入可（テストはネット非依存）。
- ・ DB 変更なしのため、本番化前の追加 SQL は発生せず（prod 適用は 0001～0004 のまま）。

9. 完了判定

Phase 6 は実質完了（実装・自動検証・CI・ローカル描画検証 すべてクリア）。

唯一の残作業は、アプリ内ボタンからの **OS 共有シート表示の端末タップ確認**（環境依存の最終目視）。これは生成ロジック・PDF 描画・ネイティブ統合が検証済みであることから、表示阻害要因はない見込み。

10. 次フェーズの提案

候補	内容・狙い	推奨度
Phase 7 組織・権限	招待フロー／ロール (owner/admin/member) ／メンバー割当。複数人運用の前提。Phase 3 で先送りした項目を回収	◎ 推奨
Phase 5 通知・連携	プッシュ通知・現場内コミュニケーション。APNs/FCM 等の外部設定が必要でセットアップ比重が大きい	○
Phase 8 品質強化	テスト網羅・クラッシュ監視・パフォーマンス。リリース品質づくり	○（後半向き）

推奨：Phase 7（組織・権限管理）。現状は単一ユーザー前提。実運用（複数人の現場）に踏み出すには、招待とロールが要。Phase 3 で意図的に外した「メンバー割当」もここで回収でき、機能の連続性が高い。着手時はこれまで同様 **要件定義書** → **承認** → **実装仕様書** → **実装** → **analyze/test/CI/実機** → **完了報告** の順で進めます。